

AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Topic 3 – AI News Briefing Service

The Three Musketeers

Ali Huseynli • Alish Hajiyeu • Elsen Samadzada

Final Project Defense – September 20, 2026

github.com/aliasefoglu/Final_Project_01

The problem in one sentence

What the system does

A user sets topic preferences once; every day, we fetch from 6 sources, remove duplicate coverage of the same story, and hand back one personalized Markdown digest with a 2–3 sentence summary and topic/sentiment label per article.

Inputs

- ▶ 5 RSS feeds + 1 scraped HTML source
- ▶ A user's preferred topics & excluded sources

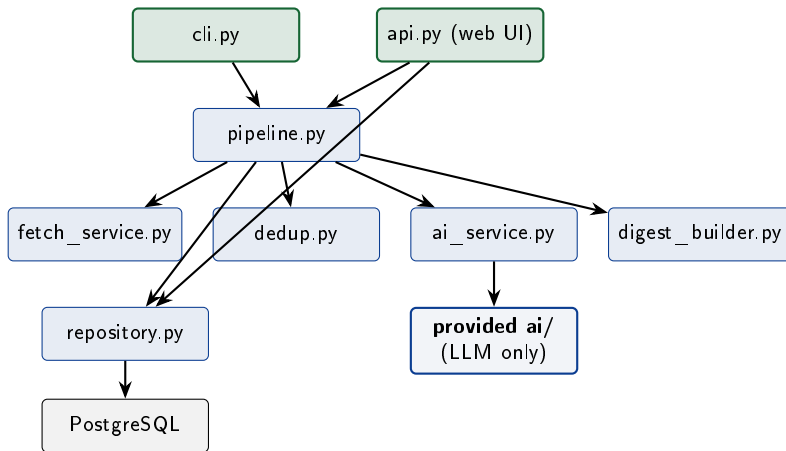
Outputs

- ▶ digests/YYYY-MM-DD-<user>.md
- ▶ A row per article in PostgreSQL (dedup cache)

Entry points: `python -m src.cli run-daily -user <name>`, plus a bonus FastAPI web UI (signup/login/preferences) not required for Topic 3.

Architecture

Module map. The boundary between the *provided* ai/ package and *our* SE layer is shown explicitly.



Three design decisions we can defend

1. **asyncio + Semaphore over threads for fetching**

Every bottleneck is I/O-bound (network waits), so asyncio's structured concurrency (gather + per-task try/except) expresses "keep going if one source fails" more directly than thread futures.

2. **Normalized join tables over a JSONB array for preferences**

Our first design stored topics as a JSONB list on users – simple, but a 1NF violation. We normalized into `user_preferred_topics/user_excluded_sources` with a composite primary key, wrapped in one transaction for atomicity.

3. **One retry policy per external boundary, not a shared one**

`fetch_service` and `ai_service` each get their own tenacity policy (different exceptions, different backoff windows) rather than one generic retry wrapper, since a fetch timeout and an LLM provider error need different handling.

Concurrency: the benchmark

Workload: `scripts/bench.py -N 20` – 20 fetch operations cycling the 6 configured sources

	Sequential (sem=1)	Concurrent (sem=8)
Wall-clock (s)	13.79	3.00
Speedup	1.0×	4.6×

Bottleneck named: per-source network RTT, plus the Reuters scrape source failing (401 Forbidden, see next slide) and retrying 3× on every single one of the 20 operations – fixed overhead concurrency can't remove.

Command: `docker compose run -rm app python scripts/bench.py -N 20`

Robustness: one concrete failure mode

Scenario (real, not staged): our Reuters HTML-scrape source returned 401 Forbidden on every request during our real benchmark run – Reuters blocks scripted requests.

How we found it: not a planned chaos test – it surfaced on its own while running `scripts/bench.py -N 20` against the real internet.

What the system did:

- ▶ Retry: tenacity retried $3\times$ with exponential backoff on every single attempt – and still failed every time.
- ▶ Isolate: `fetch_all_sources` caught it per-source; the other 5 RSS sources were unaffected.
- ▶ Log: "Source ... failed after retries; continuing without it." – full traceback preserved.
- ▶ Degradate: the benchmark still completed and reported a valid $4.6\times$ speedup using only the succeeding sources.

Live demo (or scripted demo)

What we will run

The full pipeline for one user, live: fetch → dedup → summarize → digest, printed straight to the terminal.

Demo command (works inside Docker):

```
$ docker compose run --rm app python scripts/demo.py --user khagani
```

- ▶ Watch the log lines: fetched-article count, dedup rate, then one LLM call per surviving article.

Testing

Coverage & shape

- ▶ src/ coverage: **86.7%** (90% blended w/ ai/)
- ▶ Provided ai/ smoke tests: **passing (15/15)**
- ▶ 69 tests total: 54 ours + 15 instructor smoke tests
- ▶ All 69 pass *inside Docker*, not just locally

Three tests worth calling out

- ▶ `test_run_daily_pipeline_happy_path`
- ▶ `test_fetch_all_sources_runs_sources_concurrently`
- ▶ `test_run_daily_pipeline_skips_articles_that_fail_ai_labeling`

Tooling

`pytest + pytest-asyncio + pytest-cov + respx`. **mypy: 0 errors in our code.**

Limitations & what we'd do next

Where the system is brittle today

- ▶ Confirmed, not hypothetical: Reuters actively blocks our scraper (401 Forbidden, every request).
- ▶ No multi-provider LLM failover – Gemini's free tier (5/min, 20/day) exhausted mid-run in testing; we cannot fail over to a second provider automatically.
- ▶ A known, unfixed gap: an invalid topic posted directly to /preferences raises an unhandled 500 (see report.pdf S4.3/S8).

Given another week we would

1. Fix the preferences-form validation gap above.
2. Add a real integration test against a live test PostgreSQL instance, not just a mocked Repository.

Team contributions

Member	Primary ownership
Ali Huseynli	config, models, repository, schema, CLI, tests
Alish Hajiyeu	AI service, digest builder, scheduler, web UI, Docker
Elsen Samadzada	fetch service, dedup, pipeline, benchmark, UI styling

We can all answer:

Any of us can walk through any module on demand. Pick a file — we'll defend it.

Questions?

The Three Musketeers

`github.com/aliasfoglu/Final_Project_01`

“We can walk through any file in the repo.”